

Quy trình Render 3D Gaussian Splatting và Cải tiến SADGS (SADGS)

Bản rút gọn 45 phút — Từ nền tảng đến Adaptive Density Control

Digital Twin GS

Ngày 26 tháng 9 năm 2026

Nội dung trình bày (45 phút)

- 1. Nền tảng: 3D Gaussian & Spherical Harmonics (lược gọn)
- 2. Chiếu phối cảnh & Rasterizer khả vi
- 3. Hàm mất mát, Gradient & Optimizer
- 4. **Trọng tâm:** Adaptive Density Control – cải tiến đề xuất
- 5. Ba đòn bẩy tăng tốc của SADGS
- 6. Kết quả thực nghiệm: SADGS trên Colab T4
- 7. Quan sát, giới hạn & kết luận

Phần nền tảng toán học sẽ lược gọn tối đa, dành thời gian cho đóng góp chính và số liệu thực nghiệm.

Động lực: vì sao cần một biểu diễn cảnh mới?

- NeRF: biểu diễn **implicit** bằng MLP, chất lượng cao nhưng render chậm (giây/khung hình) do ray-marching.
- 3D Gaussian Splatting (3DGS): biểu diễn **explicit** bằng hàng trăm nghìn Gaussian mờ, render bằng **rasterization khả vi**.
- Kết quả: hàng chục–hàng trăm FPS, gần thời gian thực.

Tiêu chí	NeRF	3D Gaussian Splatting
Biểu diễn	Implicit (MLP)	Explicit (tập Gaussian)
Render	Ray-marching, chậm	Rasterization, nhanh
Tốc độ	~giây/khung hình	Hàng chục–hàng trăm FPS
Huấn luyện	Backprop qua MLP	Backprop qua tham số Gaussian

3D Gaussian là gì?

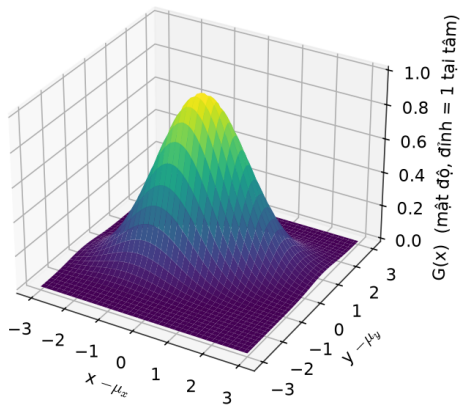
Hàm mật độ Gaussian 3D:

$$G(x) = \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

Bốn nhóm tham số học được:

- Vị trí $\mu \in \mathbb{R}^3$
- Hiệp phương sai Σ (hình dạng, hướng)
- Độ mờ (opacity) α
- Màu qua hệ số Spherical Harmonics

$$G_i(x) = \exp\left(-\frac{1}{2}(x - \mu_i)^T \Sigma_i^{-1}(x - \mu_i)\right)$$



Hình: Quả chuông mật độ Gaussian 3D

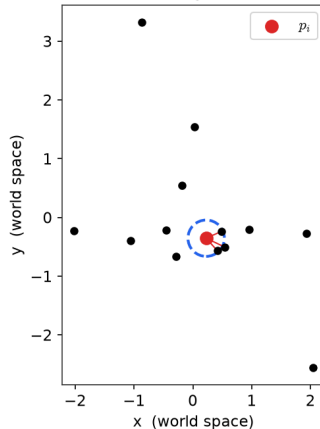
Tham số hoá hiệp phương sai

- Không học trực tiếp Σ để đảm bảo tính **positive semi-definite** khi tối ưu.
- Thay vào đó học scale $s \in \mathbb{R}^3$ và quaternion xoay $q \in \mathbb{R}^4$:

$$\Sigma = RSS^T R^T$$

với $S = \text{diag}(s)$, $R = R(q)$.

$\tilde{s}_i = \log \sqrt{d_{knn3}^2(p_i)} \cdot \vec{1}_3$
(bán kính = RMS khoảng cách tới 3 láng giềng)

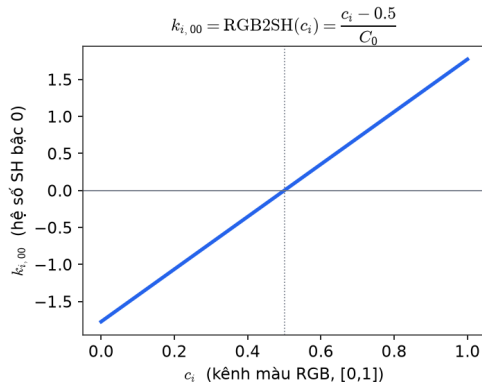


Hình: Khởi tạo scale từ khoảng cách lân cận

Màu sắc qua Spherical Harmonics (SH)

- Mỗi Gaussian không lưu 1 màu RGB cố định, mà lưu hệ số **SH**.
- Biểu diễn màu **phụ thuộc góc nhìn** (view-dependent), đặc tả hiệu ứng phản xạ/specular.
- Hệ số SH bậc 0 (DC) khởi tạo trực tiếp từ màu RGB quan sát được:

$$f_{dc} = \text{RGB2SH}(\text{color})$$

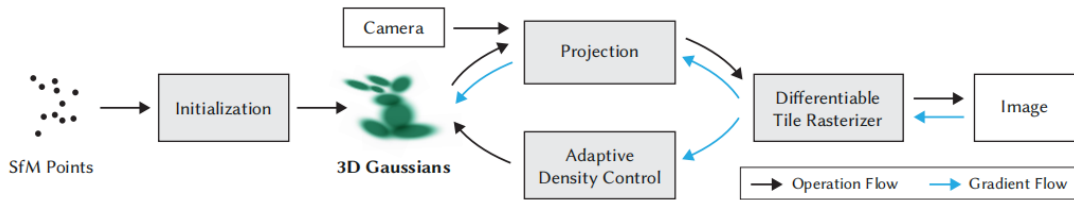


Hình: Chuyển đổi RGB $\rightarrow$ hệ số SH bậc 0

(Phần suy diễn toán học đầy đủ của SH được trình bày chi tiết trong bản đầy đủ, không đọc trong bản 45 phút.)

Tổng quan pipeline render

- World $\rightarrow$ Camera: biến đổi hệ toạ độ bằng view transform (R, t)
- Projection: chiếu các Gaussian 3D xuống mặt phẳng ảnh 2D
- Tiling + Sorting: xác định tile ảnh hưởng và sắp theo depth
- Alpha blending tuần tự $\rightarrow$ màu pixel cuối cùng



Hình: Pipeline render 3D Gaussian Splatting

Chiếu hiệp phương sai (EWA splatting)

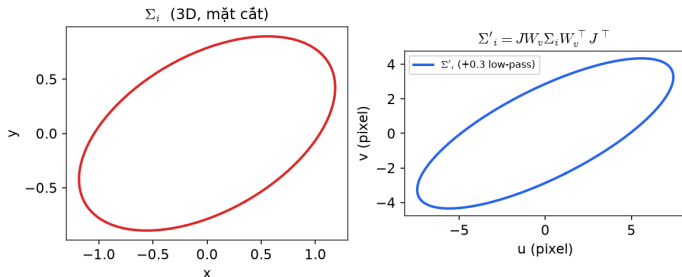
- Phép chiếu phối cảnh phi tuyến $\rightarrow$ xấp xỉ tuyến tính cục bộ bằng Jacobian J tại tâm Gaussian
- Hiệp phương sai 2D sau chiếu:

$$\Sigma' = J W \Sigma W^T J^T$$

trong đó W là phần quay của view matrix

- Kỹ thuật này gọi là **Elliptical Weighted Average (EWA) splatting**

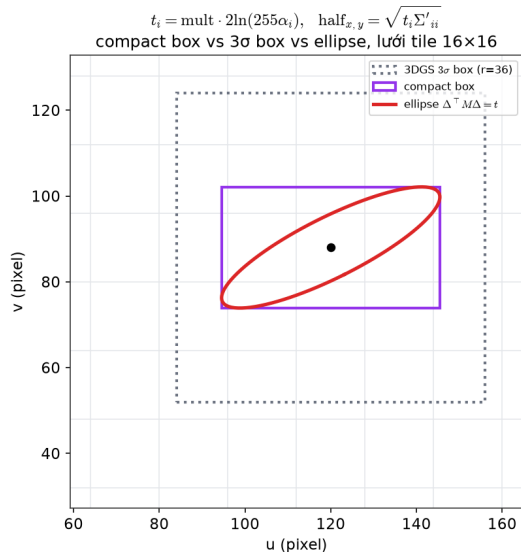
Chiếu covariance: 3D ellipsoid $\rightarrow$ 2D ellipse trên ảnh



Hình: Chiếu hiệp phương sai theo EWA splatting

Compact Box – đòn bẩy tăng tốc #1 của SADGS

- Mặc định: bounding-box 3σ – khá rộng, chạm nhiều tile không cần thiết
- SADGS thay bằng **compact box**: vùng ảnh hưởng 2D nhỏ hơn, điều chỉnh bằng hệ số `--mult`
- Kết quả: giảm đáng kể số tile mỗi Gaussian phải rasterize



Hình: So sánh bounding-box mặc định và compact box

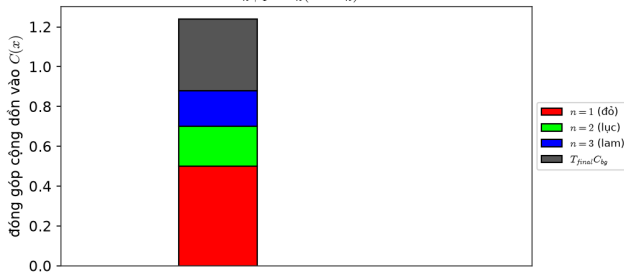
Alpha blending (front-to-back)

- Màu pixel là tổng có trọng số của các Gaussian chồng nhau theo thứ tự độ sâu
- Công thức blend:

$$C = \sum_i c_i \alpha_i T_i, \quad T_i = \prod_{j < i} (1 - \alpha_j)$$

- T_i : độ truyền sáng còn lại (transmittance) trước Gaussian thứ i

$$C(x) = \sum_n c_n \alpha_n T_n + T_{final} C_{bg}$$
$$T_{n+1} = T_n (1 - \alpha_n)$$



Hình: Chồng lớp alpha blending theo độ sâu

- Phương trình render đầy đủ:

$$C(p) = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j)$$

- Khả vi theo mọi tham số: vị trí μ , hiệp phương sai Σ , độ mờ α , hệ số SH

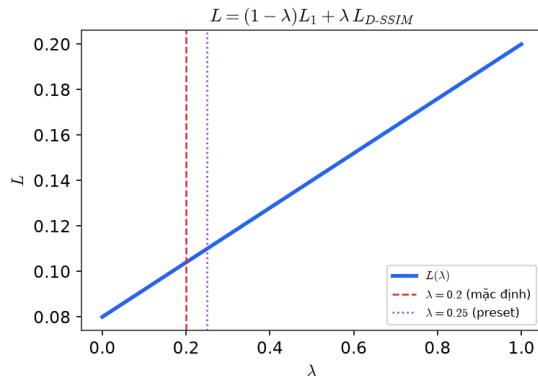
Gaussian 3D → Chiếu 2D → Trường alpha → Blend → Pixel

Hàm mất mát: kết hợp L1 và D-SSIM

- Loss huấn luyện kết hợp hai thành phần bổ trợ nhau:
 - **L1**: sai khác trung bình theo từng pixel.
 - **D-SSIM**: $1 - \text{SSIM}$, phản ánh sai lệch **cấu trúc**.
- Công thức tổng:

$$\mathcal{L} = (1 - \lambda) \mathcal{L}_1 + \lambda (1 - \text{SSIM})$$

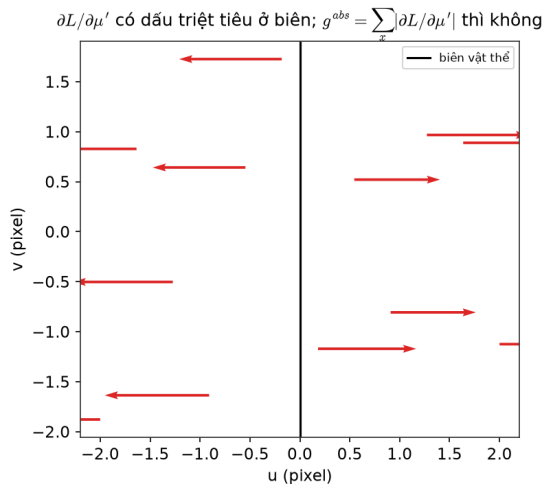
- Hệ số $\lambda = \text{lambda_dssim} = 0.2$
(arguments/__init__.py:86)



Hình: Tỷ trọng L1 và D-SSIM trong loss

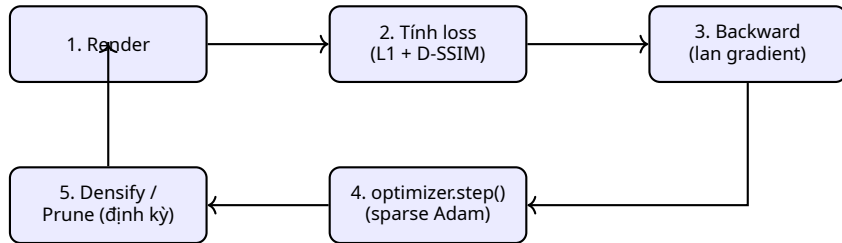
Gradient tập trung tại biên Gaussian

- Gradient view-space lớn nhất xuất hiện ở vùng **biên/rìa** của Gaussian – nơi ảnh render lệch nhiều nhất so với ảnh thật.
- Gradient tại biên chính là **tín hiệu** quyết định **densify** (thêm Gaussian mới) ở giai đoạn sau.
- Vùng gradient thấp $\Rightarrow$ Gaussian đã khớp tốt, không cần chia tách thêm.



Hình: Gradient tập trung ở biên Gaussian

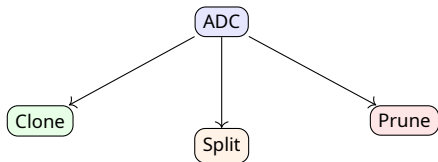
Tổng kết vòng lặp huấn luyện



- Một iteration: **render** → **loss** → **backward** → **optimizer.step()** → (định kỳ) **densify/prune** → lặp lại đến hội tụ.

Tổng quan Adaptive Density Control (ADC)

- Point cloud khởi tạo từ COLMAP thường **quá thừa** ở vùng thiếu chi tiết, đồng thời có Gaussian **quá to** che khuất chi tiết nhỏ.
- ADC chạy định kỳ mỗi `densification_interval` iteration để điều chỉnh mật độ Gaussian:
 - **Densify**: nhân bản (clone) hoặc tách (split) Gaussian thiếu hình học.
 - **Prune**: cắt tỉa Gaussian thừa, mờ, vô dụng.
- Mục tiêu: cân bằng chất lượng tái tạo và số lượng Gaussian (bộ nhớ, tốc độ render).

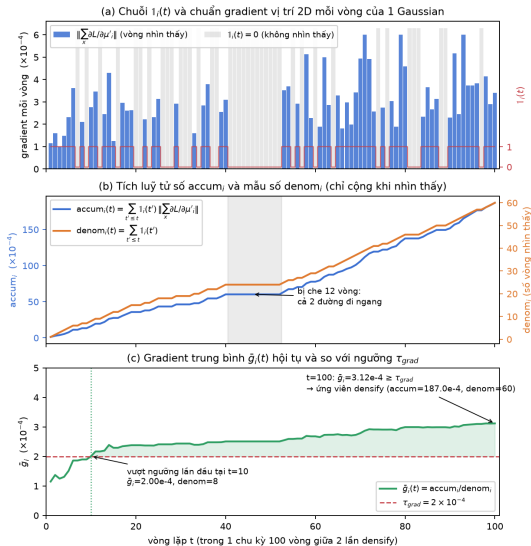


Tích lũy gradient view-space

- Với mỗi Gaussian, gradient vị trí 2D (view-space) được **cộng dồn** qua nhiều lần render từ các camera khác nhau.
- Đồng thời đếm denom : số lần Gaussian đó thực sự visible trong batch.
- Gradient trung bình dùng làm tín hiệu densify:

$$\bar{g} = \frac{\sum_v \nabla_{\text{view}}}{\text{denom}}$$

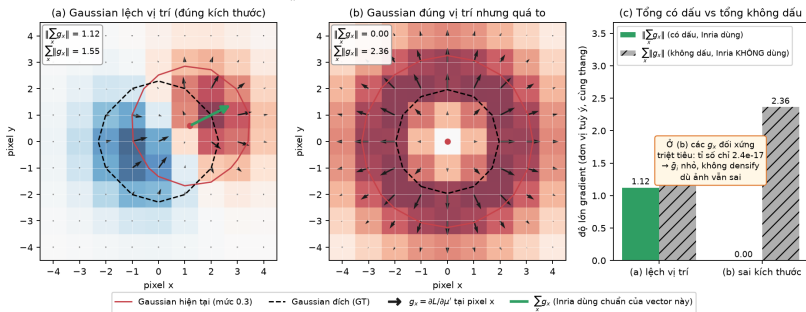
- $\bar{g}$ lớn $\Rightarrow$ ứng viên clone/split.



Vấn đề triệt tiêu gradient (gradient cancellation)

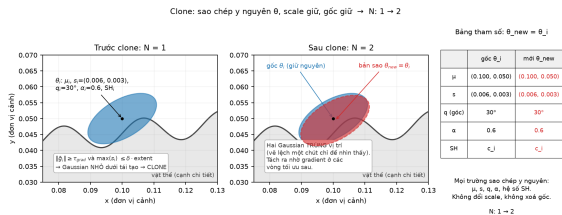
- Khi lấy trung bình vector gradient từ **những view có hướng ngược nhau**, chúng có thể **triệt tiêu lẫn nhau**.
- Hệ quả: dù mỗi view riêng lẻ báo lỗi lớn, $\bar{g}$ trung bình vẫn nhỏ $\Rightarrow$ Gaussian **bị bỏ sót**, không được densify.
- SADGS cải thiện bằng **multi-view consistency score** thay vì chỉ dựa vào gradient trung bình.

Inria chỉ tích lũy $\|\sum_x \partial L / \partial \mu'_{i,k}\|$: gradient pixel ngược chiều triệt tiêu trong kernel

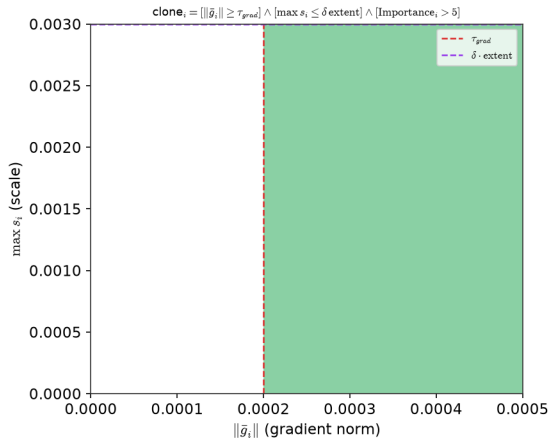


Clone (nhân bản)

- Áp dụng cho Gaussian **nhỏ nhưng thiếu hình học** (under-reconstruction).
- Nhân đôi Gaussian theo **hướng gradient vị trí $\bar{g}$** , đặt bản sao lệch một khoảng nhỏ.
- **Không** thay đổi kích thước/covariance – chỉ lấp đầy vùng thiếu chi tiết bằng tăng mật độ.



Hình: Trước/sau clone

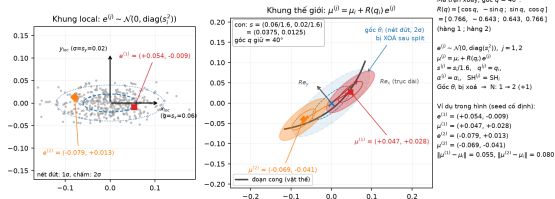


Hình: Vùng được clone

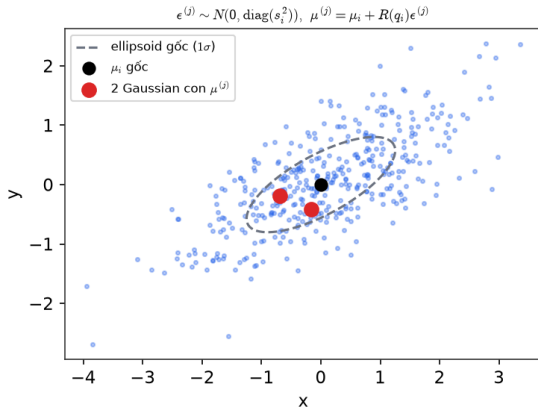
Split (tách)

- Áp dụng cho Gaussian **quá to** (over-reconstruction), vượt ngưỡng --dense theo scene extent.
- Tách thành **2 Gaussian con** nhỏ hơn (giảm scale theo hệ số cố định).
- Vị trí Gaussian con được **lấy mẫu ngẫu nhiên** theo phân phối chuẩn dựa trên covariance gốc của Gaussian cha.

Split: Gaussian TO ($\max(s_i) > \delta \cdot \text{extent}$) gán một đoạn cong $\rightarrow$ 2 con nhỏ hơn 1.6x



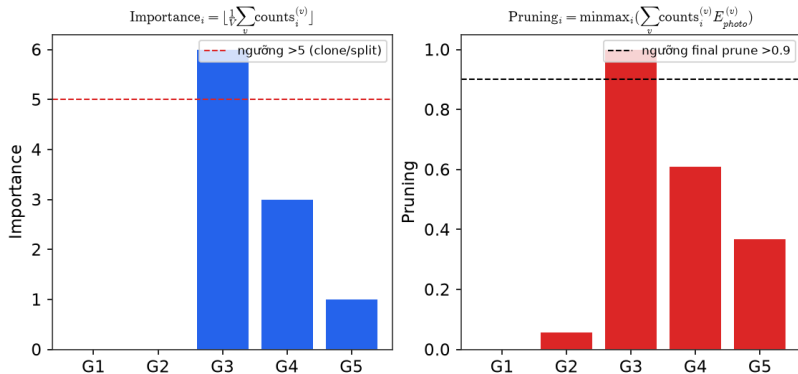
Hình: Hình học split



Hình: Lấy mẫu theo covariance

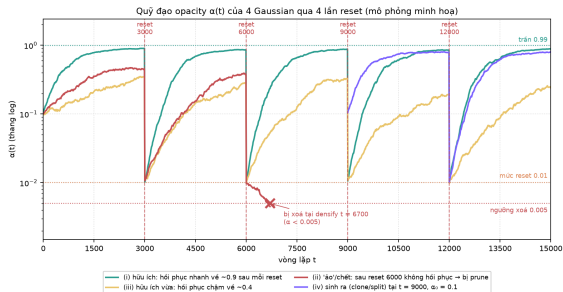
Prune theo độ quan trọng đa góc nhìn (SADGS)

- 3DGS gốc: prune đơn thuần theo ngưỡng **opacity thấp** hoặc **kích thước quá lớn**.
- **SADGS**: bổ sung tiêu chí *đa góc nhìn* – prune khi $\text{low_ratio} = \text{eta_low_count} / \text{accum_view_count} > 0.8$ (train.py:353).
- Kết quả: mô hình **gọn hơn** nhưng vẫn giữ chất lượng tái tạo – đây là **đòn bẩy chính** giúp SADGS tăng tốc.

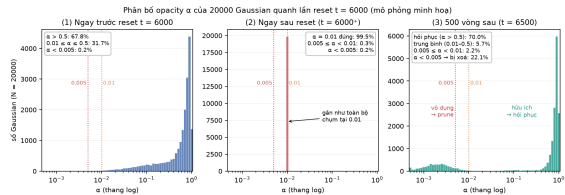


Reset opacity định kỳ

- Mỗi `opacity_reset_interval` (mặc định 3000 iteration): toàn bộ opacity reset về **giá trị thấp** (qua sigmoid-inverse).
- Buộc optimizer **"đánh giá lại"** từng Gaussian từ đầu.
- Gaussian thực sự cần thiết: opacity **tăng trở lại**; Gaussian dư thừa: không hồi phục kịp $\Rightarrow$ bị **prune** ở vòng sau.



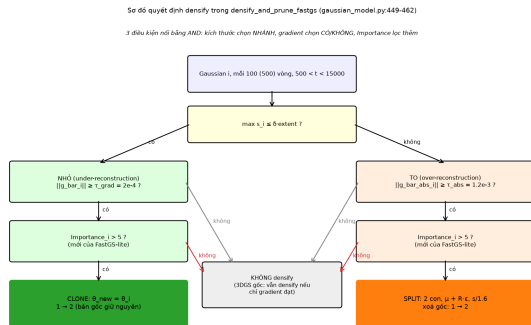
Hình: Quỹ đạo opacity theo thời gian



Hình: Histogram trước/sau reset

Sơ đồ quyết định densify đầy đủ

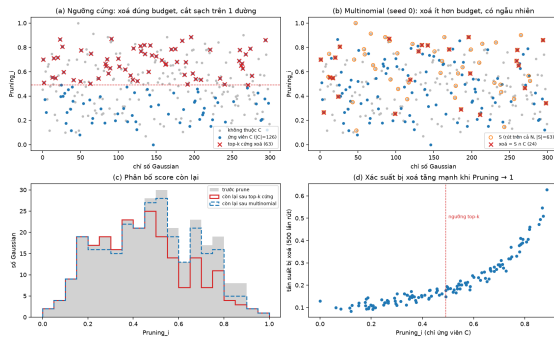
- Áp dụng cho mỗi Gaussian i , mỗi 100 (hoặc 500) vòng lặp, khi $500 < t < 15000$.
- Bước 1 — kích thước chọn nhánh:** $\max s_i \leq \delta \cdot \text{extent}$?
- Nhánh NHỎ:** nếu $\bar{g}_i \geq \tau_{\text{grad}} = 2 \times 10^{-4}$ và $\text{accum_view_count}_i > 0.5 \Rightarrow$ **CLONE** ($1 \rightarrow 2$, giữ bản gốc).
- Nhánh TO:** nếu $\bar{g}_{\text{abs},i} \geq \tau_{\text{abs}} = 2 \times 10^{-4}$ và $\text{accum_view_count}_i > 0.5 \Rightarrow$ **SPLIT** ($N = k_x k_y k_z$ con, xoá gốc).
- Nếu Importance không thỏa: 3DGS gốc vẫn densify, cơ chế này thì không (SADGS bổ sung thêm so khớp cấu trúc texture đa tỉ lệ, xem BOOK/17).
- Nguồn: `densify_and_prune_structgs`, `gaussian_model.py:957-1053`.



Hình: 3 điều kiện nối bằng AND: kích thước chọn nhánh, gradient chọn có/không, Importance lọc thêm.

Top-k cứng vs. Multinomial: hai chiến lược pruning

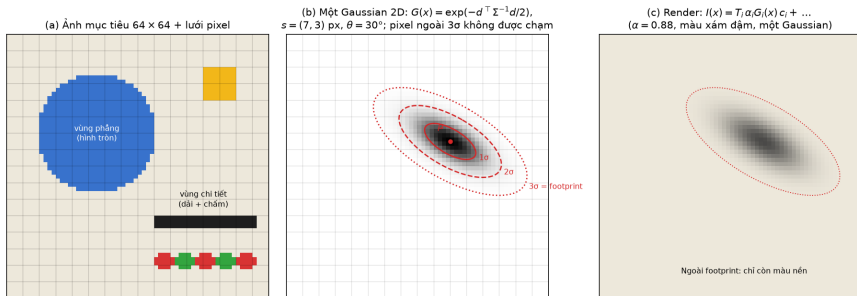
- Mô phỏng $N = 300$ Gaussian, điểm số P_i chuẩn hoá min-max từ $\text{Beta}(2,2,5)$; tập ứng viên C chọn ngẫu nhiên 40% quần thể, $\text{budget} = \lfloor 0,5|C| \rfloor$.
- **Top-k cứng:** xoá đúng budget phần tử điểm thấp nhất.
 - Cắt theo ngưỡng cứng – có thể xoá nhầm cả một cụm liên kề điểm số tương tự.
- **Multinomial (stochastic):** rút mẫu không hoàn lại theo $w_i \propto 1/(10^{-6} + 1 - P_i)$.
 - + Đa dạng hơn, tránh xoá đồng loạt một vùng.
 - Số lượng xoá thực tế thường *ít hơn* budget.



Hình: (a) top-k cứng cắt theo một đường ngưỡng; (b) multinomial xoá ít hơn budget, có ngẫu nhiên; (c) phân bố điểm còn lại; (d) xác suất bị xoá tăng mạnh khi $P_i \rightarrow 1$.

Toy model 2D: mục tiêu và một Gaussian

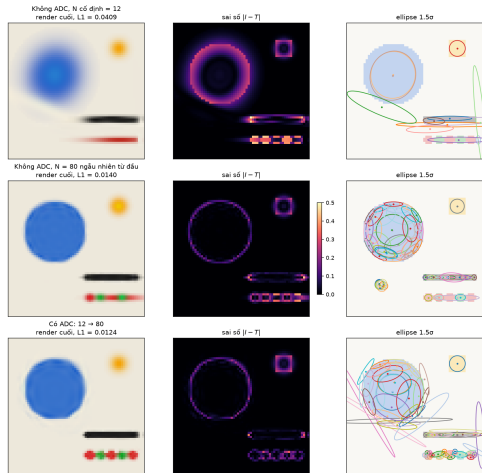
- Ảnh mục tiêu 64×64 px: hình tròn phẳng + vùng chi tiết (dải mảnh, chấm màu xen kẽ, ô vuông nhỏ).
- Mỗi Gaussian 2D: $G(x) = \exp(-\frac{1}{2}d^T \Sigma^{-1}d)$, footprint giới hạn ở 3σ .
- Render bằng alpha-compositing theo thứ tự chỉ số: $I(x) = \sum_i T_i \alpha_i G_i(x) c_i + T_{\text{end}} \cdot \text{bg}$, với $T_i = \prod_{j < i} (1 - \alpha_j G_j)$.



Hình: (a) Ảnh mục tiêu + lưới pixel; (b) một Gaussian $s = (7, 3)$ px, $\theta = 30^\circ$ với các đường mức 1σ , 2σ , 3σ ; (c) kết quả render của Gaussian đó lên nền.

So sánh trực tiếp: không ADC vs có ADC

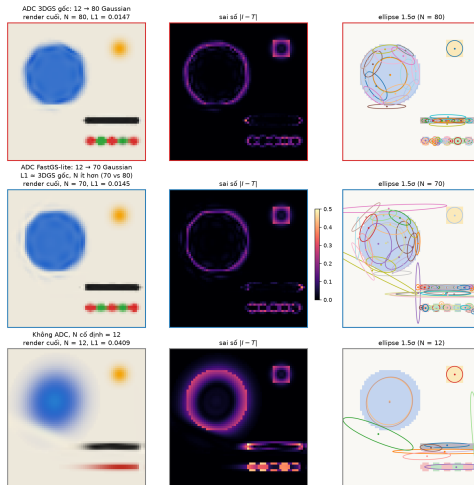
- Ba kịch bản: (1) không ADC, $N=N_0$; (2) không ADC, khởi tạo ngẫu nhiên $N=N_{\text{cuối}}$; (3) có ADC, $N_0 \rightarrow N_{\text{cuối}}$ dần.
 - (2) cùng số Gaussian (3) nhưng khởi tạo ngẫu nhiên không khớp tự nhiên vào vùng chi tiết.
- ⇒ ADC không chỉ tăng N mà đặt đúng chỗ cần thiết.



Hình: 3 hàng (không ADC N_0 ; ngẫu nhiên; có ADC) $\times$ 3 cột.

Kết quả render cuối cùng: 3 kịch bản đối chiếu

- Hàng 1 — **ADC 3DGS gốc**: N tăng từ khởi tạo lên N cuối, L_1 cuối như trong hình.
 - Hàng 2 — **ADC SADGS**: L_1 cuối **tương đương** nhưng **ít Gaussian hơn** — hiệu quả của Importance + prune multinomial + final prune.
 - Hàng 3 — **Không ADC**: $N=N_0$ cố định, không clone/split/prune — chất lượng kém hơn rõ rệt.
- Mỗi hàng: render cuối, sai số $|I - T|$, ellipse 1.5 σ . Viền màu: đỏ=3DGS, xanh=SADGS, xám=không ADC.



Hình: Đối chiếu cuối: ADC 3DGS gốc / ADC SADGS / không ADC.

- **Đang chạy trong pipeline thật:** (1) Importance đa view AND với gradient; (2) split theo gradient tuyệt đối; (3) trần opacity $\alpha \leftarrow \min(\alpha, 0.8)$ sau mỗi vòng densify+prune.
- **Không chạy trong pipeline thật** (có trong code nhưng vô hiệu): prune theo lấy mẫu multinomial – `train.py` luôn truyền `pruning_score=None`; và `final_prune_structgs` – lời gọi bị **comment out** tại `train.py:419`. Tại `prune_iterations`, code thật chỉ xoá theo $\alpha < 0.1$.

Vì sao 3DGS gốc chậm?

- Mỗi iteration gồm: **render** (rasterize), **tính loss**, **backward**, định kỳ **densify/prune**.
- Với hàng trăm nghìn – hàng triệu Gaussian, **rasterize** và **backward** chiếm phần lớn thời gian mỗi bước.

Khâu	Tần suất	Tỉ trọng thời gian
Render (project + rasterize)	mỗi iteration	~45%
Backward (gradient)	mỗi iteration	~35%
Tính loss	mỗi iteration	~10%
Densify/Prune	định kỳ	~10% (đột biến)

Ba đòn bẩy tăng tốc của SADGS (tổng quan)

1. Multi-view score

- Đánh giá Gaussian qua nhiều góc nhìn
- Pruning thông minh hơn

2. Compact Box

- Giảm hệ số nhân bounding-box
- Giảm số tile chạm

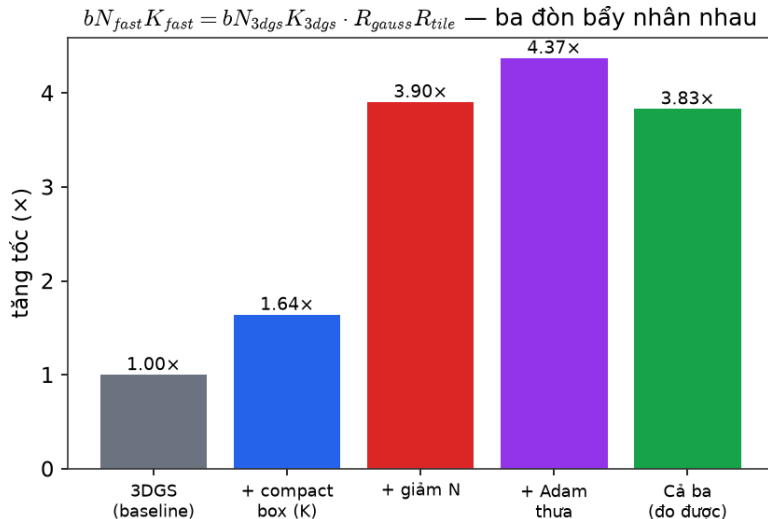
3. Sparse Adam

- Lịch cập nhật phân tầng
- Giảm tần suất optimizer.step()

- Cả ba đòn bẩy nhằm đúng khâu chiếm tỉ trọng lớn nhất: rasterize, densify/prune, cập nhật tham số.

Hiệu năng tích lũy: Biểu đồ thác nước

- Waterfall chart minh họa tăng tốc **tích lũy** khi bật lần lượt từng đòn bẫy lên baseline 3DGS gốc.
- Thứ tự: baseline → + Compact Box → + Multi-view score → + Sparse Adam → SADGS đầy đủ.



Cơ chế lỗi SADGS: densify theo cấu trúc (structure-aware)

- **Ý tưởng cốt lõi:** khớp kích thước hình chiếu của Gaussian với **bước sóng cục bộ của texture** trong ảnh – Gaussian không nên to hơn hoặc nhỏ hơn chi tiết mà nó biểu diễn.
 - **Tỉ số cấu trúc** $\eta = \frac{\text{trục Gaussian chiếu lên màn hình}}{\text{bước sóng cục bộ (structure tensor đa tỉ lệ)}}$.
 - Phân loại theo từng view, tích lũy qua nhiều view trước khi quyết định (so `high_ratio/low_ratio` với ngưỡng 0.8):
 - $\eta \geq \tau_{\text{high}} = 1.0$: Gaussian **quá to** so với chi tiết $\Rightarrow$ ứng viên **SPLIT**.
 - $\eta \leq \tau_{\text{low}} = 0.1$: Gaussian **quá nhỏ** $\Rightarrow$ ứng viên **PRUNE**.
 - **Split dị hướng theo trục** (khác với minh hoạ "tách 2 ngẫu nhiên" ở phần trước): mỗi trục x, y, z tách riêng với $k_{\text{axis}} = \lceil \sqrt{\eta_{\text{axis}}} \rceil$, sinh $N_{\text{con}} = k_x \cdot k_y \cdot k_z$ Gaussian con – mật độ con bám theo mức "quá to" thực tế của từng hướng, không cố định là 2.
- $\Rightarrow$ Hàm thực thi: `densify_and_clone_structgs, densify_and_split_structgs (gaussian_model.py)`.

Lưu ý: 2 cơ chế liên quan – `expand_undersized_gs` (giãn giải tích Gaussian dưới cỡ, $\Delta \log(\text{scale}) = -0.5 \log \eta$) và `final_prune_structgs` (prune cuối theo multi-view consistency) – được cài đặt đúng trong code nhưng lời gọi đang **bị comment out** trong `train.py`, nên **không chạy mặc định** ở lịch huấn luyện hiện tại.

Tổng kết: từ 3DGS gốc đến SADGS

Tham số	3DGS gốc	SADGS
loss_thresh	không có	mới, 0.02
grad_abs_thresh	gradient thường	Abs-GS, 0.0002
--dense	cố định	0.001 (nhân scene extent)
--highfeature_lr	lr chuẩn	0.005 (chia thêm /20)
--mult (box multiplier)	3.0	0.7

- SADGS không đổi mô hình biểu diễn cảnh, mà tối ưu **tham số hoá** và **lịch huấn luyện** $\Rightarrow$ tăng tốc mà PSNR/SSIM gần như không đổi.

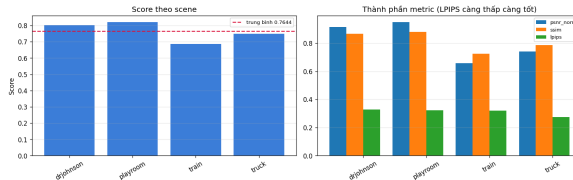
- **SADGS**: fork giữ nguyên thuật toán SADGS gốc, chỉ đóng gói lại để chạy trên Colab T4 free tier – không đổi công thức toán hay pipeline huấn luyện.

Tiêu chí	SADGS gốc	SADGS
Phần cứng mục tiêu	24GB RTX 4090	T4 (~15GB VRAM) + 12.7GB RAM
CUDA extension	git submodule	vendor sẵn trong repo (<code>git clone</code>)
Entry point	train.py + shell script	train.py + notebook có instrumentation
Tín hiệu tiến trình	L1 + EMA	Score tổng hợp, log mỗi 1000 iter

Bảng 1: Chất lượng cuối cùng theo scene

Scene	Score	PSNR (dB)	SSIM	LPIPS	#Gaussian
playroom	0.8198	-	-	-	~129k
drjohnson	0.8027	27.41	-	-	~174k
truck	0.7482	-	-	-	~187k
train	0.6867	19.75	-	-	~201k
Mean	0.7643	24.46			

- 2 scene indoor vượt 2 scene outdoor dù dùng ít **Gaussian hơn** (129k-174k so với 187k-201k).



Hình: Leaderboard Score theo scene

Bảng 3: Chi phí & lưu trữ

- Tổng train 4 scene: **327.6s** (~5.5 phút), **85.5 iter/s** trung bình, VRAM đỉnh chỉ **0.88GB** (max 1.16GB).
- .ply trung bình **171.6MB**, đúng **248 byte/Gaussian** = record 3DGS không nén.

Chỉ số	Trung bình	Ghi chú
Thời gian train (4 scene)	327.6 s	~5.5 phút tổng
Tốc độ	85.5 iter/s	
VRAM peak	0.88 GB	max 1.16GB (drjohnson)
Kích thước .ply	171.6 MB	248 byte/Gaussian

Khả năng tái lập & giới hạn phép đo

```
Config(resolution=2, iterations=7000, score_every=1000,  
       eval_views=6, psnr_max=30.0, submission_resolution=1)
```

- **Giới hạn:** chạy **một lần duy nhất** (không seed/lặp lại), chỉ **4 scene** từ 2 dataset, ngân sách 7k iteration chỉ bằng **1/3** lịch 30k thiết kế gốc; `expand_undersized_gs` và `final_prune_structgs` đang bị comment out trong `train.py` nên không chạy trong lần đo này (không riêng do ngân sách 7k).
- ⇒ **Kết luận:** Bảng 1 là *baseline có thể tái lập của repo này*, **không phải** benchmark chính thức để so sánh với SADGS/3DGS gốc.

Bảng tổng kết: 3DGS gốc (vanilla) vs. SADGS (mới)

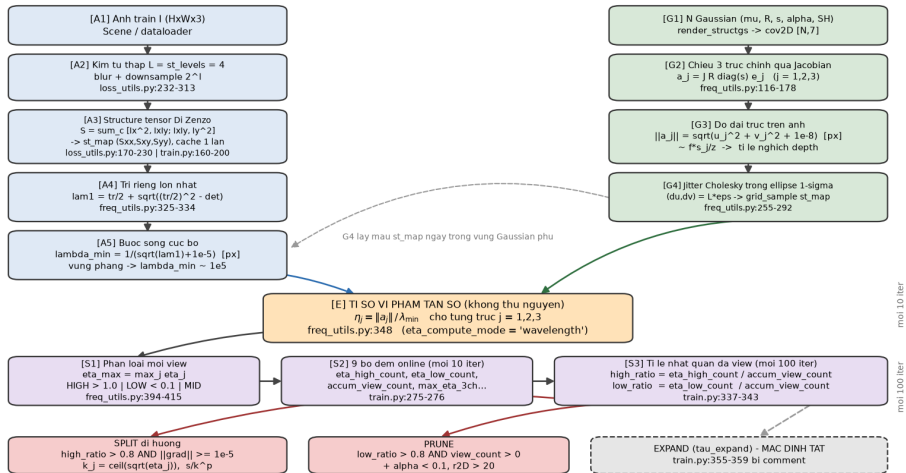
Cơ chế	3DGS gốc (densify_and_clone/split)	SADGS (mới)
Đo độ khớp kích thước	không có	$\eta = \frac{\text{trục chiếu}}{\text{bước sóng texture}}$
Ngưỡng phân loại	-	$\tau_{\text{high}}=1.0, \tau_{\text{low}}=0.1$
Split	luôn 2 con, ngẫu nhiên	dị hướng theo trục, $k_{\text{axis}} = \lceil \sqrt{\eta_{\text{axis}}} \rceil$
Giãn Gaussian dưới cỡ	không có	<code>expand_undersized_gs</code> [†]
Prune cuối	không có	<code>final_prune_structgs</code> [†]

[†] Đã cài đặt đúng trong `gaussian_model.py` nhưng lời gọi đang bị comment out trong `train.py` – không chạy mặc định (xem Slide 33, 41). Cột 3DGS gốc lấy trực tiếp từ `densify_and_clone/densify_and_split` trong `scene/gaussian_model.py`.

Bản đồ 15 slide tới: từ *structure tensor* đến ADC

SADGS: từ STRUCTURE TENSOR đến ADC

trái = nhánh ANH (1 lần, trước vòng lặp) | phải = nhánh GAUSSIAN (mỗi view)



Hình: Nhánh **ảnh** (xanh dương) chạy một lần trước vòng lặp $\rightarrow \lambda_{\min}$; nhánh **Gaussian** (xanh lá) chạy mỗi view $\rightarrow \|a_j\|$. Hai nhánh gặp nhau đúng một chỗ: $\eta_j = \|a_j\| / \lambda_{\min}$ (ô cam). Tím = bỏ phiếu đa view, hồng = ADC, xám nét đứt = mặc định TẮT.

Ba điều cần nhớ trước khi đi vào chi tiết

- 1 η là tỉ số không thứ nguyên (pixel/pixel) — nhờ vậy hai ngưỡng hằng $\tau_{high}=1.0$, $\tau_{low}=0.1$ mới dùng được xuyên cảnh.
- 2 Giữ ba η , không phải một — ba trục chính của ellipsoid, nhờ đó mới chia *dị hướng* được: $k_j = \lceil \sqrt{\max(\eta_j, 1)} \rceil$.
- 3 Đo tách khỏi quyết định — đo mỗi 10 iteration, quyết định mỗi 100 iteration bằng `high_ratio/low_ratio` so với 0.8.

Cổng AND (`train.py:345`) là chốt chặn chống bùng nổ N :

$$\text{split} = (\text{high_ratio} > 0.8) \wedge (\|\nabla_{\mathbf{x}_{2D}} \mathcal{L}\| \geq 10^{-5})$$

(a) TRUOC: eta = (4.0, 1.3, 0.2)
1 Gaussian det, vì phạm mảnh theo trục 1



(b) 3DGS: split dạng hương
 $N = 2$, mỗi trục đều chia 1.6



(c) 5ADGS: $k_j = \text{ceil}(\sqrt{\eta_j}) = (2, 2, 1)$
 $N_{con} = k_x * k_y * k_z = 4$, $s_j \rightarrow s_j / k_j^p$



Đọc tu code: `gaussian_model.py:701` chỉ clamp0s, min=1) - KHÔNG có tran 8 như comment hui; chốt chặn duy nhất là ngưỡng gradient 1e-5 (`train.py:333`).

Hình: 3DGS chia đều mọi trục cho 1.6; SADGS chia riêng từng trục theo η_j rồi thu scale s_j/k_j^p .

Tensor cấu trúc đa tỉ lệ — nền tảng đo cấu trúc ảnh của SADGS

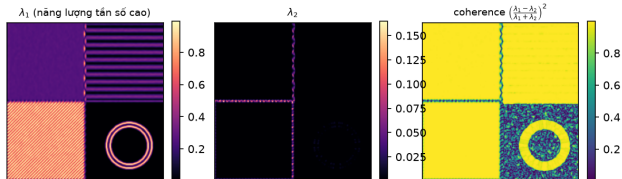
$$J_{\rho} = G_{\rho} * \begin{bmatrix} \sum_c I_{x,c}^2 & \sum_c I_{x,c} I_{y,c} \\ \sum_c I_{x,c} I_{y,c} & \sum_c I_{y,c}^2 \end{bmatrix}$$

$$\lambda_{1,2} = \frac{\text{tr}}{2} \pm \sqrt{\left(\frac{\text{tr}}{2}\right)^2 - \det}, \quad w_{\min} = \frac{1}{\sqrt{\lambda_1}}$$

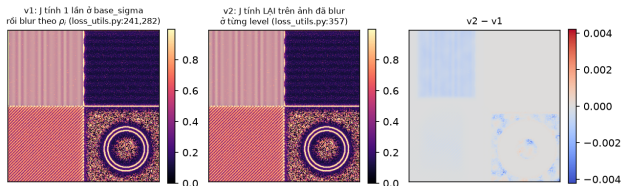
$$\hat{J} = \frac{\sum_i \frac{J_i}{w_i} b_i^p f_i^2}{\sum_i b_i^p + 10^{-6}}, \quad \sigma_i = \sigma_0 s^i, \quad f_i = s^{-i}$$

- Sobel sau làm mượt, cộng năng lượng 3 kênh RGB (Di Zenzo) — `loss_utils.py:170--230`.
- $\lambda_1 \rightarrow$ bước sóng nhỏ nhất (px); $v_2 \perp v_1 \rightarrow$ hướng texture.
- Đa tỉ lệ: b_i là DoG, $\rho_i = 3\sigma_i$; mặc định `levels=4`, $\sigma_0=1$, $s=1.5$, $p=3$.
- **v1** (:282) làm mờ lại tensor gốc; **v2** (:357) tính lại tensor trên ảnh đã mờ ở từng thang. Mặc định `st_mode="v1"`.
- `st_map` (1, 3, H, W) = (S_{xx} , S_{xy} , S_{yy}), cache theo `image_name` một lần trước khi train (`train.py:160--200`).

$\lambda_{1,2} = \frac{\text{tr}}{2} \pm \sqrt{\left(\frac{\text{tr}}{2}\right)^2 - \det}$ — SADGS dùng λ_1 làm ngưỡng bước sóng $w_{\min} = 1/\sqrt{\lambda_1}$ (`freq_utils.py:325-334`)



So sánh trace $S_{xx} + S_{yy}$ của `st_map` (levels=4). Mặc định train: `st_mode="v1"` (`arguments/_init_.py:121`)



Hình: Trên: λ_1 , λ_2 , coherence. Dưới: trace `st_map` theo v1 vs v2.

SADGS — Tỷ số cấu trúc η : Gaussian có "quá to" so với texture?

1. Chiều 3 trục xuống màn hình

(freq_utils.py:116--178)

$$A_{cam} = R_{view}R(r) \text{diag}(s), \quad \begin{aligned} u_j &= J_{00}A_j^x + J_{02}A_j^z \\ v_j &= J_{11}A_j^y + J_{12}A_j^z \end{aligned}$$

với j có hàng $u = (f_x/z, 0, -f_x x/z^2)$, hàng $v = (0, f_y/z, -f_y y/z^2)$; depth z , tâm means2D lấy từ cov2D của rasterizer.

$$\|a_j\| = \sqrt{u_j^2 + v_j^2 + 10^{-8}} \text{ [pixel]} \propto fs_j/z$$

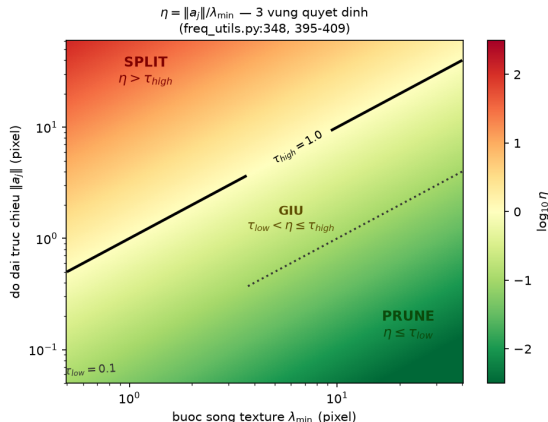
2. Bước sóng cục bộ (fu:325-334)

$$\lambda_1 = \frac{\text{tr}}{2} + \sqrt{\left(\frac{\text{tr}}{2}\right)^2 - \det}, \quad \lambda_{\min} = \frac{1}{\sqrt{\lambda_1 + 10^{-5}}}$$

3. Tỷ số (eta_compute_mode="wavelength", fu:348)

$$\eta_j = \|a_j\|/\lambda_{\min} \quad j = 1, 2, 3 \text{ (3 trục chính)}$$

$\eta_{\max} > \tau_{\text{high}} = 1.0 \Rightarrow \text{SPLIT}$; $\eta_{\max} \leq \tau_{\text{low}} = 0.1 \Rightarrow \text{PRUNE}$
(fu:395-409), chốt khi tỉ lệ đa view > 0.8 (train.py:345,353).



Hình: η trên lưới (độ dài trực chiều) $\times$ (bước sóng texture): 3 vùng SPLIT / GIỮ / PRUNE. Trục giác Nyquist — Gaussian rộng hơn chu kỳ texture thì xoá mất chi tiết.

SADGS — Tỷ lệ nhất quán đa view cho densify/prune

η (mức vi phạm tần số) phụ thuộc view $\Rightarrow$ **không** quyết định trên 1 view. Mỗi 10 iteration, `update_freq_stats_online` (`utils/freq_utils.py:181`) tích lũy bộ đếm cho từng Gaussian, chỉ với mẫu qua bộ lọc

$$(T_{\max} > 0) \wedge (\alpha > 0.05) \wedge (\|\nabla_{xy}\| > 2 \times 10^{-5})$$

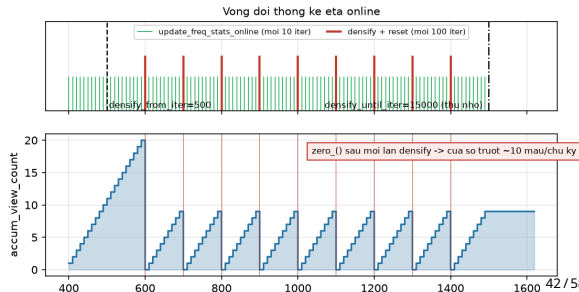
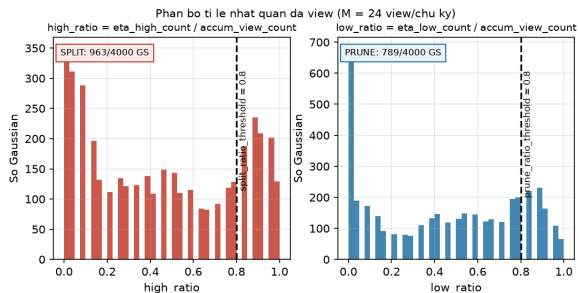
Phân loại mỗi quan sát theo $\eta_{\max} = \max_k \eta_k$: HIGH nếu $> \tau_{\text{high}}=1.0$, LOW nếu $\leq \tau_{\text{low}}=0.1$ (`freq_utils.py:395--409`).

Mỗi `densification_interval=100` iter (`train.py:337--353`):

$$r^{\text{high}} = \frac{\text{eta_high_count}}{\text{accum_view_count}}, \quad r^{\text{low}} = \frac{\text{eta_low_count}}{\text{accum_view_count}}$$

$$\text{SPLIT} : r^{\text{high}} > 0.8 \wedge \|\bar{g}\| \geq 10^{-5}; \quad \text{PRUNE} : r^{\text{low}} > 0.8$$

Kích thước con định hình bằng `max_eta_3ch` (worst-case qua view), không dùng trung bình. Sau densify, cả 9 bộ đếm `zero_()` (`train.py:377--386`) $\Rightarrow$ cửa sổ trượt ≈ 10 mẫu/chu kỳ.



SADGS: tách Gaussian **dị hướng** theo từng trục (densify_and_split_structgs)

Ba công thức cốt lõi (scene/gaussian_model.py:638--832)

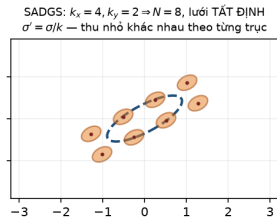
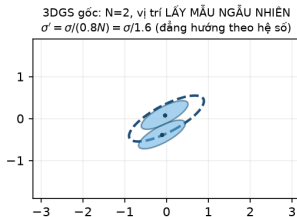
$$k_a = \max\left(1, \left\lceil \sqrt{\max(\eta_a, 1)} \right\rceil\right) \quad (699-701)$$

$$N = k_x k_y k_z \quad (714), \quad \sigma'_a = \frac{\sigma_a}{k_a^p} \quad (722)$$

$$\mathbf{x}_{\text{con}} = \mathbf{x}_{\text{cha}} + R(q) \left[\sqrt{12} \frac{\sigma}{k} \odot \left(\mathbf{i} - \frac{\mathbf{k}-1}{2} \right) \right] \quad (763-788)$$

- η_a = năng lượng tần số vượt Nyquist trên trục $a \Rightarrow$ **trục nào aliasing nhiều thì cắt nhiều**, trục đủ mịn giữ $k = 1$.
- Con nằm trên **lưới đều tất định** trong hệ local rồi xoay bằng $R(q)$; hệ số $\sqrt{12}$ giữ phương sai cha.
- Kế thừa nguyên vẹn rotation, SH, **opacity không giảm**, tmp_radii, filter_3D; densify_count+1. Cha bị prune_points xóa sau khi thêm con (807-831).
- So với 3DGS gốc (:866--892): luôn $N=2$, tâm **ngẫu nhiên** $\mathcal{N}(0, \Sigma)$, $\sigma' = \sigma/1.6$ cho **mọi** trục.
- **Chú thích dòng 698** nói "clamp max=8" nhưng code chỉ `clamp(min=1)` — k không có trần.

Cùng một Gaussian dị hướng — hai cách tách



Hình: Cùng một Gaussian dị hướng. Trái: 3DGS gốc, 2 con ngẫu nhiên, vẫn dài theo trục vi phạm. Phải: SADGS $k_x=4, k_y=2 \Rightarrow N=8$ con trên lưới tất định, mỗi trục co đúng hệ số cần thiết.

Clone trong SADGS và cách ghép tensor vào optimizer

1. Chọn điểm (gaussian_model.py:935, 998--1007):

$$S = \underbrace{(\text{acc_view_cnt} > 0.5)}_{\text{metric_mask}} \wedge (\text{eta_mask} \vee \|\bar{g}\| \geq \tau_g) \wedge \left(\max_i s_i \leq \lambda_d \text{ ext} \right)$$

$\tau_g = 2 \times 10^{-4}$, $\lambda_d = \text{args.dense} = 0.001$ (arguments:112--113). Khác 3DGS (:895--897): tiêu chí đến từ ngoài, có nhánh tần số đa view, ngưỡng scale chặt hơn $10\times$.

2. Sao chép nguyên xi (:937--944): $\mu_{\text{new}} = \mu_{\text{old}}$, không dịch vị trí, không chia scale; densify_count kế thừa (:952--954).

3. Ghép vào Adam (:578--586):

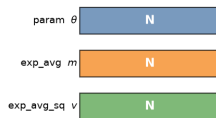
$$\theta' = \text{cat}(\theta, \theta[S]), \quad m' = \text{cat}(m, \mathbf{0}_M), \quad v' = \text{cat}(v, \mathbf{0}_M)$$

Phải del opt.state[p] → tạo nn.Parameter mới → gán lại state, vì khoá của opt.state chính là đối tượng Parameter.

4. Hệ quả reset moment: bước đầu của điểm mới $\approx \frac{1-\beta_1}{\sqrt{1-\beta_2}} = 3.16$ lần bước thường — chính cú hích này tách hai bản sao trùng khít ra khỏi nhau.

cat_tensors_to_optimizer: ghép M Gaussian mới vào Adam (566-594)

TRƯỚC (N Gaussian)



SAU (N+M Gaussian)



$\theta' = \text{cat}(\theta, \theta[S]) \quad m' = \text{cat}(m, \mathbf{0}_M) \quad v' = \text{cat}(v, \mathbf{0}_M)$
del opt.state[p] -> group["params"][0] = nn.Parameter(cat) -> opt.state[p_new] = state
(gaussian_model.py:578-586 — phải dựng lại param group vì id(Parameter) là khoá của opt.state)

Hình: Chiều tensor $N \rightarrow N + M$: tham số được sao chép, hai moment Adam khởi tạo 0 (cat_tensors_to_optimizer, :566--594).

Giãn giải tích Gaussian dưới cỡ (expand_undersized_gs) — và trạng thái *bị tắt*

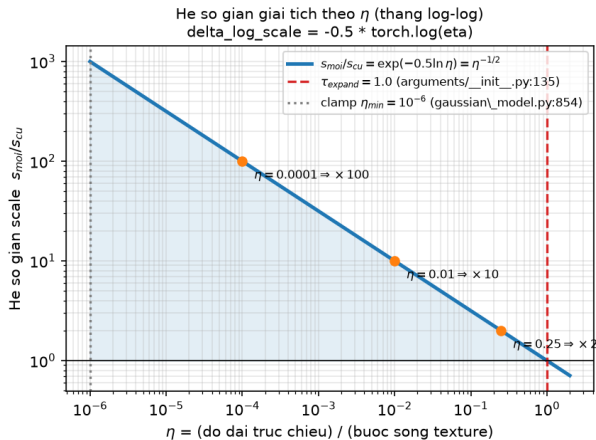
Gaussian có $\eta = \ell_{\text{truc}} / \lambda_{\min} < \tau_{\text{expand}}$ là **nhỏ hơn** chi tiết texture $\Rightarrow$ lãng phí điểm. SADGS sửa **trực tiếp** tham số thay vì sinh thêm Gaussian:

$$\text{mask} = (\eta < \tau_{\text{expand}}) \wedge (\eta > 0), \quad \tau_{\text{expand}} = 1.0$$

$$\Delta_{\log} = -\frac{1}{2} \ln(\max(\eta, 10^{-6})) \Rightarrow \frac{s_{\text{mới}}}{s_{\text{cũ}}} = \eta^{-1/2}$$

- Cộng thẳng vào log-scale vì $\sigma = \exp(\text{_scaling})$; N **không đổi** $\Rightarrow 0$ byte thêm (so với split cần k^3 Gaussian, $k = \lceil \eta^{-1/2} \rceil$).
- Ghi qua `replace_tensor_to_optimizer(..., "scaling")` $\Rightarrow$ **xoá moment Adam của toàn bộ** param group.
- η trong code là **tuyến tính** theo scale, nên một bước chỉ đưa $\eta \rightarrow \sqrt{\eta}$, không phải 1 như docstring khẳng định.
- **Lỗi gọi trong `train.py:356--359` đang bị COMMENT OUT** — mặc định SADGS chỉ SPLIT/PRUNE.

`gaussian_model.py:833--864`, `train.py:355--359`, `arguments/\_\_init\_\_.py:135`, `freq_utils.py:348`



Hình: Hệ số giãn $\eta^{-1/2}$ theo η (log-log): chỉ vùng $\eta < \tau_{\text{expand}}$ bị tác động; clamp 10^{-6} giới hạn giãn ở $\approx 1000\times$.

SADGS: densify_and_prune_structgs – một vòng sinh + xoá hợp nhất

Mỗi 100 iteration (từ iter 500), `train.py`:362 gọi một hàm duy nhất.
Thứ tự thật trong code (`gaussian_model.py`:957--1053):

CLONE → SPLIT → PRUNE → trần $\alpha \leq 0.8$

Mật nạ sinh (`dense = 0.001`):

$$\text{clone} = (S \vee Q) \wedge [s_{\max} \leq \text{dense} \cdot \text{ext}]$$

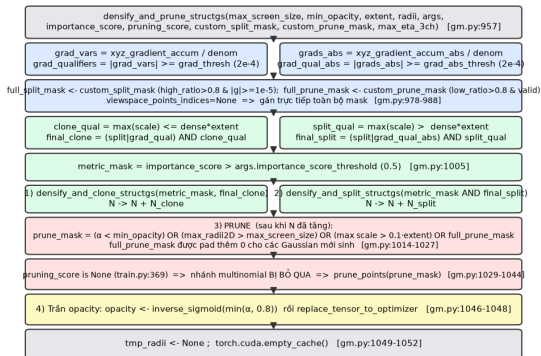
$$\text{split} = (S \vee Q_{\text{abs}}) \wedge [s_{\max} > \text{dense} \cdot \text{ext}]$$

Mật nạ xoá là **hợp OR** của 4 tiêu chí:

$$[\alpha < 0.1] \vee [r_{2D} > 20] \vee [s_{\max} > 0.1 \text{ ext}] \vee [\text{low_ratio} > 0.8]$$

- `min_opacity` thật = **0.1** (`train.py`:364), gấp $20 \times$ mức 0.005 của 3DGS gốc.
- r_{2D} và $s_{\max}$ chỉ xét sau iter 3000 (trước đó `max_screen_size=None`).
- `low_ratio` được pad 0 cho Gaussian vừa sinh $\Rightarrow$ chúng miễn nhiệm tiêu chí multi-view ở chính vòng này.
- `pruning_score=None` $\Rightarrow$ nhánh multinomial 50% không chạy: xoá cứng toàn bộ ứng viên.
- `prune_points` dùng `valid =  $\neg$ mask` cắt đồng bộ 6 tham số, mọi bộ đếm η , và `exp_avg/exp_avg_sq` của **cả** optimizer lẫn `shoptimizer`.

Thứ tự THỰC SỰ trong code: **CLONE → SPLIT → PRUNE → trần opacity 0.8**



Hình: Luồng thật của `densify_and_prune_structgs`: prune luôn chạy **sau** khi N đã tăng, nên Gaussian con mới sinh cũng bị soi bởi ngưỡng opacity/bán kính/scale.

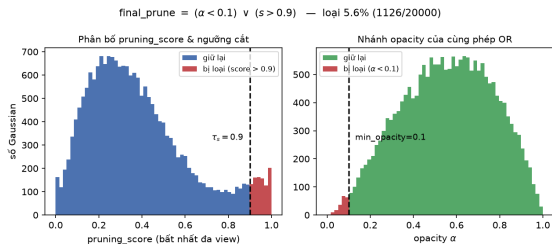
Tỉa cuối final_prune_structgs: công thức có, lời gọi đang tắt

Toàn bộ hàm (scene/gaussian_model.py:1059--1066) là một phép OR tắt định:

$$\text{prune}_i = [\alpha_i < \tau_\alpha] \vee [s_i > \tau_s]$$

với $\tau_\alpha = \text{min_opacity} = 0,1$ (lời gọi train.py:419) và $\tau_s = 0,9$ **hard-code** ở :1064; s_i là pruning_score - độ *bất nhất* tái dựng qua nhiều view. Mặt nạ đi thẳng vào prune_points (:528), cắt cả tham số lẫn mọi bộ đếm η .

- Chạy tại prune_iterations mặc định [4000, 8000] (train.py:412, 529).
- Khác prune trong vòng density: ở đó có remove_budget = int(0.5*to_remove) + multinomial (:1029--1042); ở đây cắt dứt khoát, không phanh.
- **Lời gọi bị comment** (train.py:418--419); nguồn điểm số compute_gaussian_score_structgs **không tồn tại** trong repo; mặc định pruning_score=None sẽ gây TypeError vì :1064 không kiểm tra None.
- Cái thực chạy chỉ còn $[\alpha_i < 0,1]$ (train.py:416--417) $\Rightarrow$ floater (opacity cao, chỉ đúng vài view) lọt lưới, mô hình cuối to hơn cần thiết.



Hình: Hai nhánh của phép OR: $\tau_s = 0,9$ chỉ cắt đuôi bất nhất, $\tau_\alpha = 0,1$ cắt opacity yếu (code ghi rõ # 0.005 is a good default, train.py:364).

Optimizer, lịch learning rate & reset opacity trong SADGS

1. Lịch lr vị trí (get_expon_lr_func, general_utils.py:32--64):

$$\text{lr}(k) = \exp\left[(1-t) \ln \text{lr}_{\text{init}} + t \ln \text{lr}_{\text{fin}}\right], \quad t = \text{clip}\left(\frac{k}{30000}, 0, 1\right)$$

với $\text{lr}_{\text{init}} = 1.6 \times 10^{-4} \cdot s$, $\text{lr}_{\text{fin}} = 1.6 \times 10^{-6} \cdot s$. lr_delay_steps không được truyền (= 0) nên warm-up **không kích hoạt**.

2. lr hằng số các nhóm khác (training_setup:306--313): opacity 0.05, scaling 0.01, rotation 0.002, $f_{\text{dc}} 0.0025$, $f_{\text{rest}} = 0.005/20$ - ba nhóm đầu **gấp đôi** 3DGS gốc.

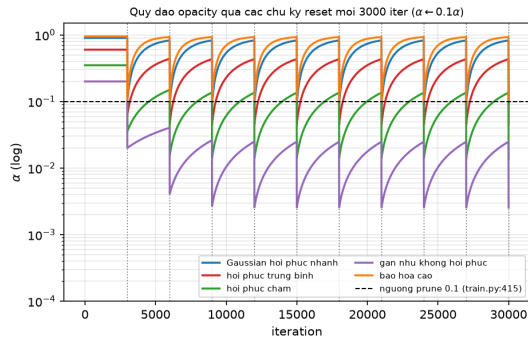
3. Optimizer hybrid (mặc định): Adam+amsgrad cho hình học $\oplus$ SparseGaussianAdam cho SH bậc cao (chỉ cập nhật Gaussian có radii>0), $\varepsilon = 10^{-8}$.

4. Reset opacity mỗi 3000 iter (train.py:402):

$$\alpha_{\text{new}} = \frac{\sigma(o) \cdot c \cdot 0.1}{c} = 0.1 \sigma(o)$$

hệ số 3D filter c **triệt tiêu** $\Rightarrow$ chỉ là phép **nhân** 0.1; khác 3DGS gốc kẹp $\min(\alpha, 0.01)$ - SADGS **giữ nguyên thứ tự** độ đục rồi prune ở ngưỡng 0.1.

5. Bậc SH: oneupSHdegree gọi **mỗi iteration** $\Rightarrow d_k = \min(k, 3)$, bảo hoà ngay tại $k = 3$ (3DGS: mỗi 1000 iter).



Hình: Quỹ đạo α (log) qua 10 chu kỳ reset $\alpha \leftarrow 0.1\alpha$; đường đứt là ngưỡng prune 0.1.

Bộ lọc 3D chống alias: chặn dưới kích thước Gaussian theo Nyquist

1. Ngưỡng lọc (`gaussian_model.py:254`) — z là **độ sâu** tới camera *gần nhất* nhìn thấy Gaussian (:245), $f = \max_i f_x^{(i)}$ của camera phân giải cao nhất (:247):

$$\text{filter_3D} = \frac{z}{f} \sqrt{0.2} \approx 0.447 \times (\text{1 pixel quy về 3D})$$

2. Scale hiệu dụng (:156--161) — tích chập Gaussian*Gaussian, $\Sigma \rightarrow \Sigma + \sigma_f^2 I$:

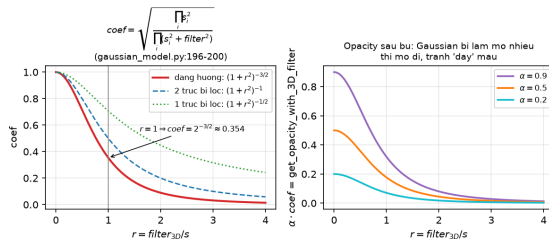
$$s_i^{\text{eff}} = \sqrt{s_i^2 + \text{filter_3D}^2} \xrightarrow{s_i \rightarrow 0} \sigma_f$$

3. Bù opacity (:190--201) — bảo toàn năng lượng $\alpha \sqrt{\det \Sigma}$:

$$\text{coef} = \sqrt{\frac{\prod_i s_i^2}{\prod_i (s_i^2 + \sigma_f^2)}}, \quad \alpha^{\text{eff}} = \alpha \cdot \text{coef}$$

đẳng hướng: $\text{coef} = (1 + r^2)^{-3/2}, r = \sigma_f/s$.

- Dùng ở render: `opacity` (`gaussian_renderer:63`), `scales` (:74). Bật bằng `--compute_3d_filter` (mặc định `False`, arguments:132); tắt $\Rightarrow$ `filter_3D=0` (`train.py:158`).
- Cập nhật: 1 lần trước vòng lặp (`train.py:155`), sau đó mỗi **100 iteration** nhưng **chỉ sau** `densify_until_iter=15000` và dừng trước 100 bước cuối (`train.py:405--409`). Con sinh ra khi `densify` **kế thừa** `filter_3D` của cha (:728, :886, :907); `prune` lọc theo mask (:547--548).
- Với η : SADGS chia $k_i = \lceil \sqrt{\eta_i} \rceil$ để làm s nhỏ đi; bộ lọc 3D là **phanh** — khi $s \lesssim \sigma_f$, s^{eff} bão hoà còn `coef` sụp $\Rightarrow$ con quá nhỏ mất opacity và bị `prune` (`train.py:416`).



Hình: Hệ số bù opacity theo $r = \text{filter_3D}/s$: Gaussian nhỏ hơn pixel bị mờ đi thay vì nhấp nháy.

Hàm mất mát của SADGS: $L_1 + \text{SSIM} + L_2$ (và hai loss chưa dùng)

Loss thực sự được tối ưu — `train.py:259`:

$$\mathcal{L} = (1 - \lambda_{\text{dssim}})L_1 + \lambda_{\text{dssim}}(1 - \text{SSIM}) + \lambda_{L_2}L_2$$

$$= 0.8L_1 + 0.2(1 - \text{SSIM}) + 2.0L_2$$

$\lambda_{\text{dssim}} = 0.2$, $\lambda_{L_2} = 2.0$ (`arguments/__init__.py:86,117`). SSIM dùng kernel CUDA fused_ssim (`train.py:18,258`), không dùng `ssim()` thuần torch ở `loss_utils.py:46` (cửa sổ Gaussian 11×11 , $\sigma = 1.5$, $C_1 = 0.01^2$, $C_2 = 0.03^2$).

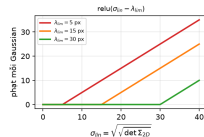
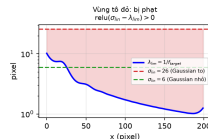
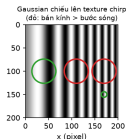
Hai loss được định nghĩa nhưng KHÔNG vào $\mathcal{L}$:

- tone_curve_loss (:28): $\frac{1}{N} \sum \left(\frac{p - g}{\text{sg}(p) + \varepsilon} \right)^2$, $\varepsilon = 10^{-6}$ — chia sai số cho độ sáng dự đoán (stop-gradient) $\Rightarrow$ học trên sai số tương đối, khuếch đại gradient vùng tối.
- frequency_loss (:80): $\frac{1}{N} \sum \text{ReLU}(\sigma_{\text{lin}} - \lambda_{\text{lim}})$ với $\sigma_{\text{lin}} = \sqrt{\det \Sigma_{2D}}$ và $\lambda_{\text{lim}} = 1/\sqrt{S_{xx} + S_{yy}}$ lấy mẫu từ `st_map` tại `means2D` — phạt Gaussian to hơn bước sóng texture.

Cả hai có trọng số = 0 và không được tham chiếu ở bất kỳ đâu (`lambda_tone`, `lambda_freq` — `args:118-119`).

$\Rightarrow$ Loss của SADGS = loss 3DGS gốc + một số hạng L_2 trọng số lớn. Cơ chế "structure-aware" nằm ở densification (thống kê η không gradient, `freq_utils.py:181`) chứ không nằm trong hàm mất mát.

`loss_utils.py:80-110` — `frequency_loss(means2D, cov2D, st_map, H, W)`



Hình: `frequency_loss` (`loss_utils.py:80-110`): bước sóng cho phép λ_{lim} cơ lại ở vùng texture rậm; Gaussian có σ_{lin} vượt ngưỡng bị phạt tuyến tính một phía.

Lấy mẫu camera bằng FPS & lấy mẫu Gaussian dị hướng 2D

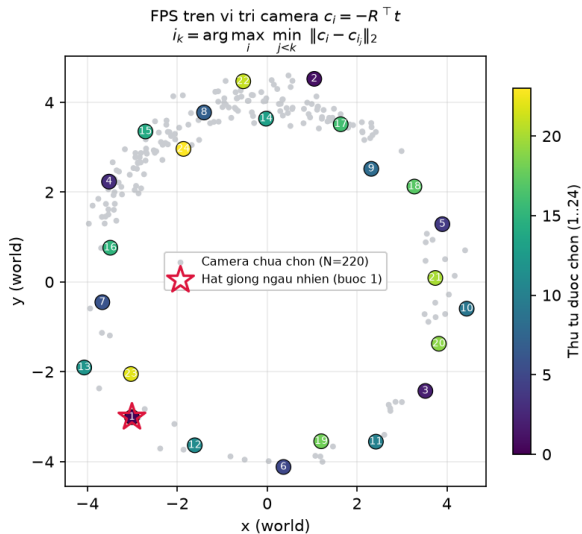
`sampling_cameras` (utils/freq_utils.py:12--87), mặc định mode="fps",
num_cams=60. Vị trí camera $c_i = -R^\top t$ (dòng 47), rồi tham lam:

$$d_i \leftarrow \min(d_i, \|c_i - c_{i_{k-1}}\|_2), \quad i_k = \arg \max_i d_i$$

Metric chỉ dùng VỊ TRÍ, không dùng hướng nhìn. Chi phí $O(N \cdot \text{num_cams})$
 $\Rightarrow$ phải lấy tập con để thống kê tần số η đủ rẻ.

`sample_anisotropic_gaussians_2d` (utils/gaussian_sampling.py:11): vị
trí $\sim p \propto S_{xx} + S_{yy}$; hình dạng từ $\lambda_{1,2} = \frac{\text{tr} S}{2} \pm \sqrt{(\frac{\text{tr} S}{2})^2 - \det S}$, trục
dài dọc v_2 (hướng cạnh).

Sự thật cần nêu: hàm này *không* được gọi ở đâu; khởi tạo point cloud vẫn
là `create_from_pcd` từ COLMAP (scene/__init__.py:83). Và ở
`train.py:224`, 240 FPS được gọi với `num_cams=len(stack)` nên chỉ thay
thế `random.shuffle`.



Hình: FPS trên tập vị trí camera: màu/số là thứ tự chọn — mẫu trải
đều dù mật độ camera gốc rất lệch.

Siêu tham số & lịch trình huấn luyện của SADGS

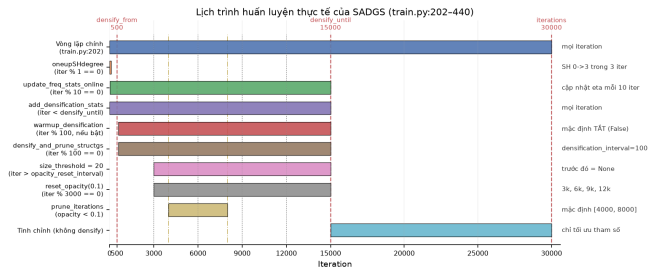
Các cờ MỚI quan trọng nhất (arguments/__init__.py)

Cờ	Mặc định
mult	0.7
eta_compute_mode	"wavelength"
split_ratio_threshold	0.8
prune_ratio_threshold	0.8
dense	0.001
freq_grad_threshold	2e-5
freq_opacity_threshold	0.05
opacity_reset_decay	0.1
highfeature_lr	0.005 (/20)
lambda_l2	2.0
optimizer_type	"hybrid"
warmup_densification	False

$\tau_{high} = 1.0, \tau_{low} = 0.1$ **hard-code** trong freq_utils.py:395--396.

Lịch trình: SH lên bậc mỗi iter; cập nhật η mỗi 10 iter (đến 15k); densify mỗi 100 iter (500–15k) rồi xoá bộ đếm η ; reset opacity mỗi 3k ($\alpha \leftarrow 0.1\alpha$); prune thô tại 4k/8k; 15k–30k chỉ tinh chỉnh.

Bị comment-out: expand_undersized_gs ($\Rightarrow$ tau_expand vô hiệu),
prune warmup, final_prune_structgs.



Hình: Lịch trình huấn luyện thật dựng từ train.py: 202--440.

Rasterizer riêng của SADGS: mult, cov2D và $T^{\max}$

SADGS fork rasterizer CUDA của 3DGS thành diff-gaussian-rasterization_structgs, gọi qua render_structgs(..., mult, ...) (gaussian_renderer/__init__.py:18). **1. Nền tảng không đổi** — tile

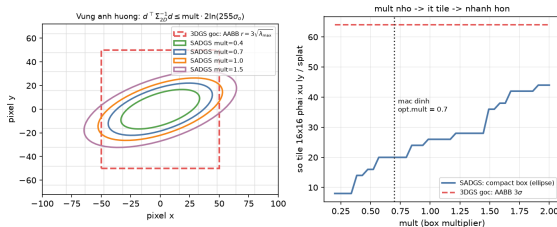
16×16 (config.h:16--17), sort theo [tile|depth], alpha blending:

$$\alpha_i = \sigma_{o,i} e^{-\frac{1}{2} d_i^T \Sigma_{2D,i}^{-1} d_i}, \quad T_i = \prod_{j < i} (1 - \alpha_j), \quad C = \sum_i c_i \alpha_i T_i$$

bỏ qua $\alpha < 1/255$, dừng sớm khi $T < 10^{-4}$. **2. mult** — compact box thay AABB 3σ : chỉ gán tile cho vùng

$$d^T \Sigma_{2D}^{-1} d \leq t, \quad t = \text{mult} \cdot 2 \ln(255 \sigma_o)$$

(auxiliary.h:332--333). Mặc định opt.mult = 0.7 (arguments/__init__.py:114) $\Rightarrow$ ít cặp (tile, Gaussian) $\Rightarrow$ sort ngắn, render nhanh; phần bị cắt có α dưới cả ngưỡng $1/255$ nên chất lượng gần như không đổi.



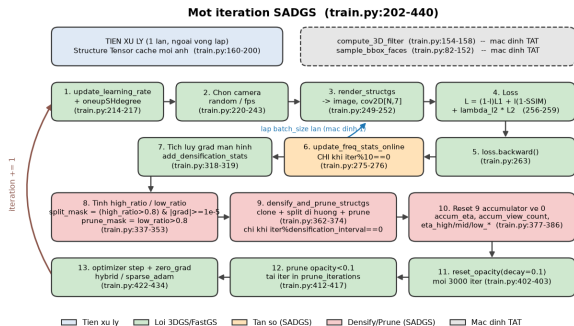
Hình: Vùng ảnh hưởng theo mult so với AABB 3σ và số tile 16×16 phải xử lý mỗi splat.

3. Đầu ra mới: cov2D ($N, 7$) (rasterize_points.cu:92)

- cột 0-2: Σ_{2D} ; cột 3-4: tâm pixel; cột 5: depth (forward.cu:241--246)
- cột 6: $T^{\max}$ ghi bằng atomicMax trên bit-pattern float (forward.cu:470)

Python lọc ($T^{\max} > \tau_T$) $\wedge$ ($\sigma_o > \tau_o = 0.05$) rồi lấy mẫu Cholesky từ Σ_{2D} để tính η (utils/freq_utils.py:181--260). Kèm theo: accum_metric_counts, depth/opacity/normal_map khi compute_extra.

Tổng hợp đầu-cuối: pipeline SADGS, chi phí, và phần chưa bật



Hình: 13 bước của một iteration SADGS (train.py:202--440); khối xám nét đứt = mặc định TAT.

Chi phí mỗi vòng:

$$t_{\text{iter}} = aN + bP + c + \frac{a_{\eta}N}{10} + \frac{a_D N}{100}$$

Freq stats chỉ chạy khi $\text{iter} \% 10 == 0$ (train.py:275); densify chỉ khi $\text{iter} \% 100 == 0$. Vì t tuyến tính theo N , **prune đa view** ($\text{low_ratio} > 0.8$) là đòn bẩy mạnh nhất.

Khác biệt cốt lõi so với 3DGS gốc

(densify_and_clone/densify_and_split, scene/gaussian_model.py): tín hiệu densify là AND của gradient và $\text{high_ratio} > 0.8$; split *dị hướng* $k_{\text{axis}} = \lceil \sqrt{\eta_{\text{axis}}} \rceil$; prune thêm tiêu chí đa view. Nền rasterization (compact box mult = 0.7, Sparse Adam) giữ nguyên.

Đã cài nhưng KHÔNG chạy: expand_undersized_gs (train.py:356--359), final_prune_structgs (418--419), prune warmup (393--400), training_report (306), scene.save (442) — đều bị comment; cộng 10 cờ mặc định False và 6 tham số dead code.

Rủi ro do đặc: render.py:40--44 đo FPS không `cuda.synchronize()`; metrics.py:74 dùng SSIM khác hàm với lúc train; full_eval.py:90,95,121 truyền `--budget/--mode/--sh_lower` chưa được khai báo trong arguments.

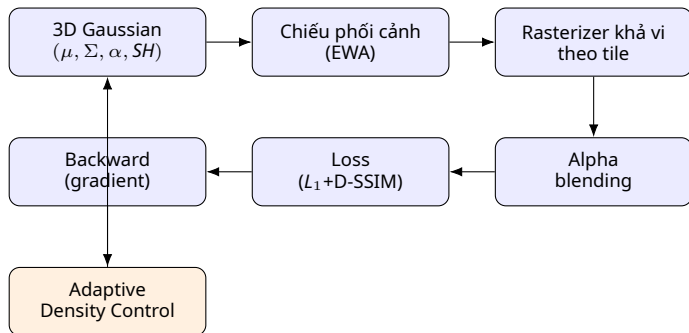
Quan sát 2: Nút thắt là RAM hệ thống, không phải VRAM

- Đo trên Google Colab T4 trong quá trình huấn luyện:

Tài nguyên	Đỉnh sử dụng	Khả dụng	% sử dụng
VRAM (GPU)	1.16 GB	14.56 GB	8%
RAM hệ thống	9.59 GB	12.7 GB	91% (giới hạn mềm 10.5 GB)

- RAM hệ thống áp sát giới hạn mềm, trong khi VRAM còn dư rất nhiều.
- Khuyến nghị cũ "giảm độ phân giải để tiết kiệm VRAM" **không cần thiết** ở ngân sách này.

Tổng kết pipeline render 3D Gaussian Splatting



Một chu trình khép kín: render → tính loss → lan truyền ngược → điều chỉnh mật độ Gaussian → lặp lại.

Tổng kết cải tiến SADGS & giới hạn còn lại

Đóng góp cốt lõi + các đòn bẩy hiệu năng:

- **Structure-aware densification** (đóng góp chính, Slide 33): so khớp kích thước Gaussian với bước sóng texture qua tỉ số η ; **split dị hướng theo trục** ($k_{axis} = \lceil \sqrt{\eta_{axis}} \rceil$) thay vì luôn tách 2 như 3DGS gốc $\Rightarrow$ mật độ Gaussian bám sát độ chi tiết ảnh thay vì đồng nhất.
- **Multi-view score + Compact box** $\Rightarrow$ giảm Gaussian dư thừa mà không giảm chất lượng đáng kể.
- **Sparse Adam** $\Rightarrow$ giảm chi phí tính toán mỗi iteration.

Giới hạn / hướng phát triển:

- 1 `expand_undersized_gs` (giãn giải tích Gaussian dưới cỡ) và `final_prune_structgs` (prune cuối theo multi-view consistency) **đã cài đặt đúng** trong `gaussian_model.py` nhưng lời gọi **đang bị comment out** trong `train.py` $\Rightarrow$ không chạy trong lịch huấn luyện mặc định, kể cả khi tăng ngân sách iteration.
- 2 Dữ liệu Gaussian đầu ra chưa được nén (248 B/Gaussian, thô).
- 3 Chưa có ablation study cô lập riêng từng đòn bẩy.
- 4 4 nhánh mở rộng của SADGS gốc — dynamic scenes, sparse view, surface reconstruction, SLAM — chưa tích hợp vào fork này.

Cảm ơn đã theo dõi!

Hỏi đáp (Q&A)

Digital Twin GS — SADGS
github.com/KietAnhCS/SADGS